Architecture 1001: x86-64 Assembly ShiftExample3Signed.c

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

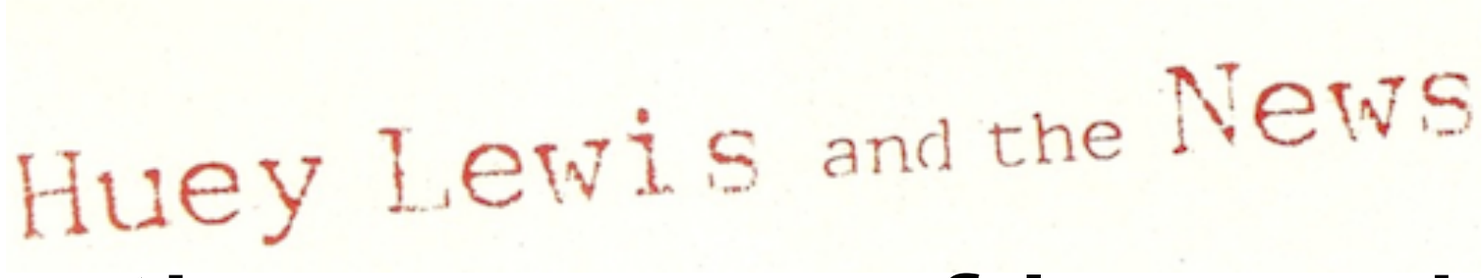
to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"



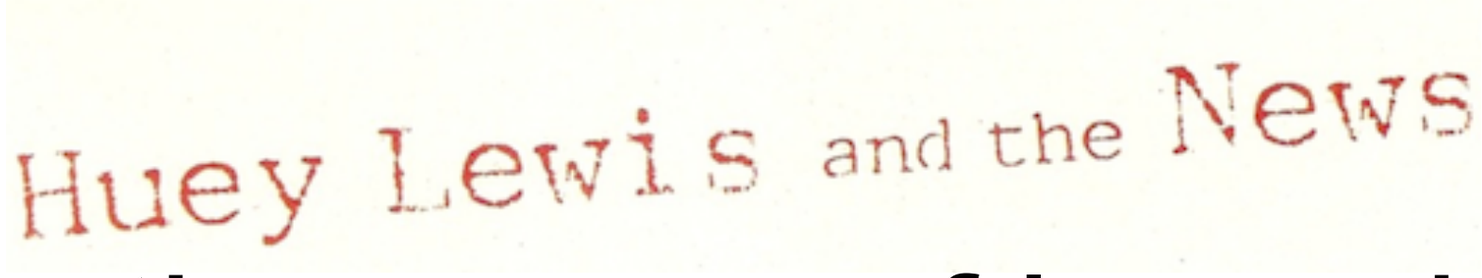
That's the power of ~~love~~ sign!

```c
#include <stdlib.h>

int main(int argc, char **argv)
{
    unsigned int a, b, c;
    a = atoi(argv[0]);
    b = a * 8;
    c = b / 16;
    return c;
}
```

```c
#include <stdlib.h>

int main(int argc, char **argv)
{
    int a, b, c;
    a = atoi(argv[0]);
    b = a * 8;
    c = b / 16;
    return c;
}
```



That’s the power of ~~love~~ sign!

```c
#include <stdlib.h>

int main(int argc, char **argv)
{
    unsigned int a, b, c;
    a = atoi(argv[0]);
    b = a * 8;
    c = b / 16;
    return c;
}
```

```c
#include <stdlib.h>

int main(int argc, char **argv)
{
    int a, b, c;
    a = atoi(argv[0]);
    b = a * 8;
    c = b / 16;
    return c;
}
```

"The power of sign, is a curious thing!
Make one man weep, make another man sing!"
It's about integer overflow vulnerabilities! :D

ShiftExample3Signed.c

Note: Compiled with /O2 “Maximize Speed”, to clear away a bit of cruft

```
#include <stdlib.h>

int main(int argc, char **argv)
{
    int a, b, c;
    a = atoi(argv[1]);
    b = a * 8;
    c = b / 16;
    return c;
}
```

```
main:
0000000140001000  sub  rsp,28h
0000000140001004  mov  rcx,qword ptr [rdx]
0000000140001007  call qword ptr [0000000140004178h]
000000014000100D  shl  eax,3
0000000140001010  cdq
0000000140001011  and  edx,0Fh
0000000140001014  add  eax,edx
0000000140001016  sar  eax,4
0000000140001019  add  rsp,28h
000000014000101D  ret
```

ShiftExample3Signed.c

Note: Compiled with /O2 “Maximize Speed”, to clear away a bit of cruft

```
#include <stdlib.h>

int main(int argc, char **argv)
{
    int a, b, c;
    a = atoi(argv[1]);
    b = a * 8;
    c = b / 16;
    return c;
}
```

```
main:
0000000140001000  sub  rsp,28h
0000000140001004  mov  rcx,qword ptr [rdx]
0000000140001007  call qword ptr [0000000140004178h]
000000014000100D  shl  eax,3
0000000140001010  cdq
0000000140001011  and  edx,0Fh
0000000140001014  add  eax,edx
0000000140001016  sar  eax,4
0000000140001019  add  rsp,28h
000000014000101D  ret
```

ShiftExample3Signed.c

Note: Compiled with /O2 “Maximize Speed”, to clear away a bit of cruft

```c
#include <stdlib.h>

int main(int argc, char **argv)
{
    int a, b, c;
    a = atoi(argv[1]);
    b = a * 8;
    c = b / 16;
    return c;
}
```

```
main:
0000000140001000  sub  rsp,28h
0000000140001004  mov  rcx,qword ptr [rdx]
0000000140001007  call qword ptr [0000000140004178h]
000000014000100D  shl  eax,3
0000000140001010  cdq
0000000140001011  and  edx,0Fh
0000000140001014  add  eax,edx
0000000140001016  sar  eax,4
0000000140001019  add  rsp,28h
000000014000101D  ret
```

```
main:
0000000140001010  sub           rsp,28h
0000000140001014  mov           rcx,qword ptr [rdx]
0000000140001017  call          qword ptr [40008368h]
000000014000101D  shl           eax,3
0000000140001020  shr           eax,4
0000000140001023  add           rsp,28h
0000000140001027  ret
```

Vs

```
main:
0000000140001010  sub           rsp,28h
0000000140001014  mov           rcx,qword ptr [rdx]
0000000140001017  call          qword ptr [40008368h]
000000014000101D  shl           eax,3
0000000140001020  cdq
0000000140001021  and           edx,0Fh
0000000140001024  add           eax,edx
0000000140001026  sar           eax,4
0000000140001029  add           rsp,28h
000000014000102D  ret
```

Changed

CDQ is added as an VS-ism. It's necessary for the math to work out, but I feel like I've only run into it *once ever* in the wild. So I don't consider it that important for beginners to know and I'm skipping it. But you can feel free to come back and read this code once we've gone through the RTFM section.

SAR - Shift Arithmetic Right

- Can be explicitly used with the C “>>” operator, if operands are signed
- First operand (source and destination) is an r/mX
- Second operand is either cl (lowest byte of ecx), or a 1 byte immediate. The 2nd operand is the number of places to shift
- It divides the register by 2 for each place the value is shifted. More efficient than a divide instruction
- Each bit shifted off the right side is placed in CF

sar bl, 2

	10110011b (bl - 0xB3)
result	**11**101100b (bl - 0xEC)

!=

shr bl, 2

	10110011b (bl - 0xB3)
result	**00**101100b (bl - 0x2C)

mov cl, 2; sar bl, cl

	00110011b (bl - 0x33)
result	**00**001100b (bl - 0x0C)

==

mov cl, 2; shr bl, cl

	00110011b (bl - 0x33)
result	**00**001100b (bl - 0x0C)

SAL - Shift Arithmetic Left

- **Actually behaves exactly the same as SHL!**
- First operand (source and destination) is an r/mX
- Second operand is either cl (lowest byte of rcx), or a 1 byte immediate. The 2nd operand is the number of places to shift
- It multiplies the register by 2 for each place the value is shifted. More efficient than a multiply instruction
- Each bit shifted off the left side is placed in CF

sal bl, 2

	10110011b (bl - 0xB3)
result	11001100b (bl - 0xCC)

==

shl bl, 2

	10110011b (bl - 0xB3)
result	11001100b (bl - 0xCC)

mov cl, 2; sal bl, cl

	00110011b (bl - 0x33)
result	11001100b (bl - 0xCC)

==

mov cl, 2; shl bl, cl

	00110011b (bl - 0x33)
result	11001100b (bl - 0xCC)



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

CHECK YOUR UNDERSTANDING



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

CHECK YOUR UNDERSTANDING

ShiftExample3Signed.c Takeaways

- Compilers still prefer shifts for mul/div over powers of 2
- But when the operands are *signed* rather than unsigned, it must use different instructions, and potentially do more work to deal with a signed multiply
- CDQ isn't super important for beginners to know, left as an exercise for the reader to RTFM for later ;)

```c
#include <stdlib.h>

int main(int argc, char **argv)
{
    int a, b, c;
    a = atoi(argv[1]);
    b = a * 8;
    c = b / 16;
    return c;
}
```

```
main:
0000000140001010  sub  rsp,28h
0000000140001014  mov  rcx,qword ptr [rdx]
0000000140001017  call qword ptr [40008368h]
000000014000101D  shl  eax,3
0000000140001020  cdq
0000000140001021  and  edx,0Fh
0000000140001024  add  eax,edx
0000000140001026  sar  eax,4
0000000140001029  add  rsp,28h
000000014000102D  ret
```

Instructions we now know (26)

- NOP
- PUSH/POP
- CALL/RET
- MOV
- ADD/SUB
- IMUL
- MOVZX/MOVSX
- LEA
- JMP/Jcc (family)
- CMP/TEST
- AND/OR/XOR/NOT
- INC/DEC
- SHR/SHL/**SAR/SAL**

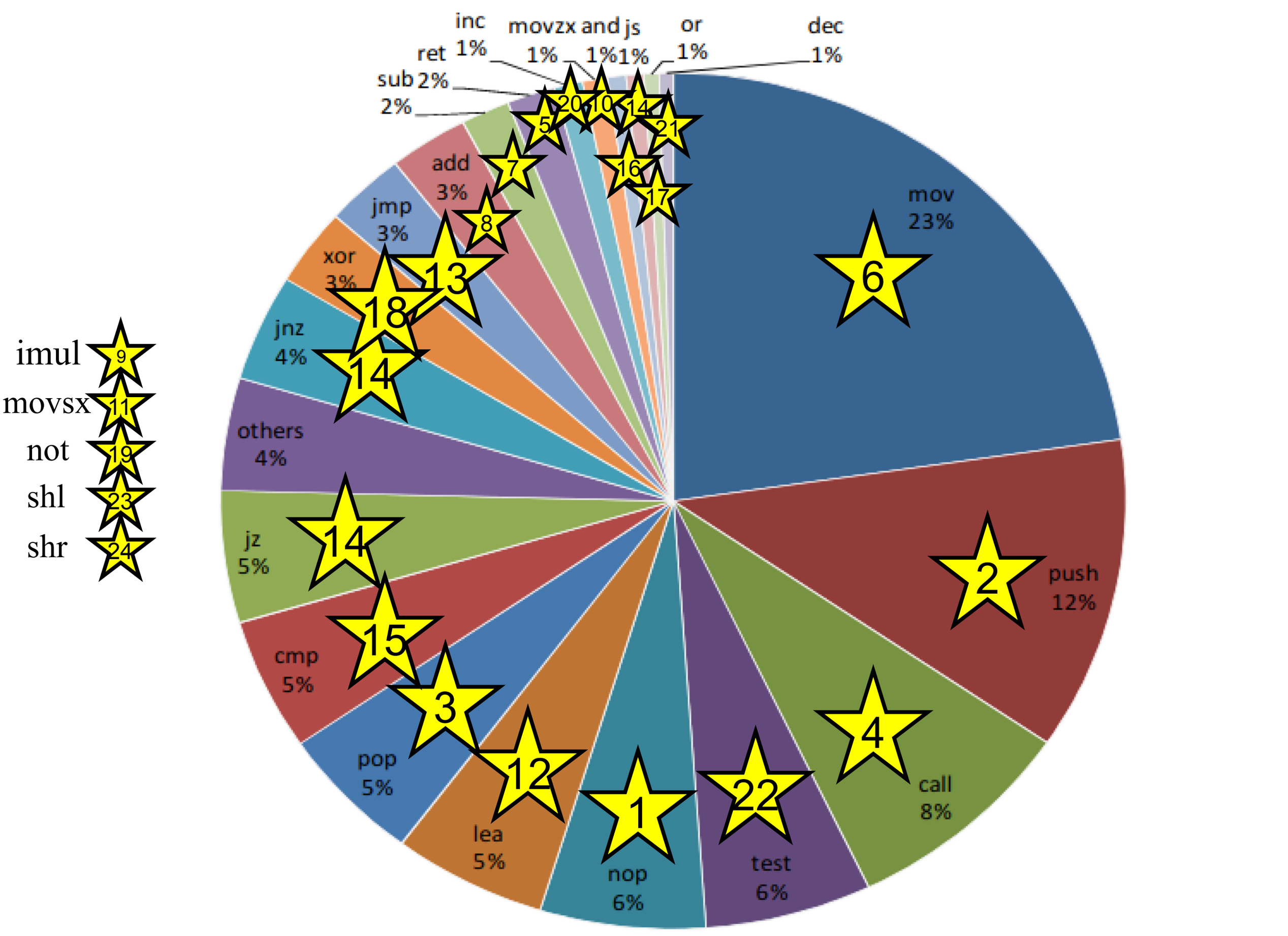


Fig. 2 Web browsers' instruction frequencies

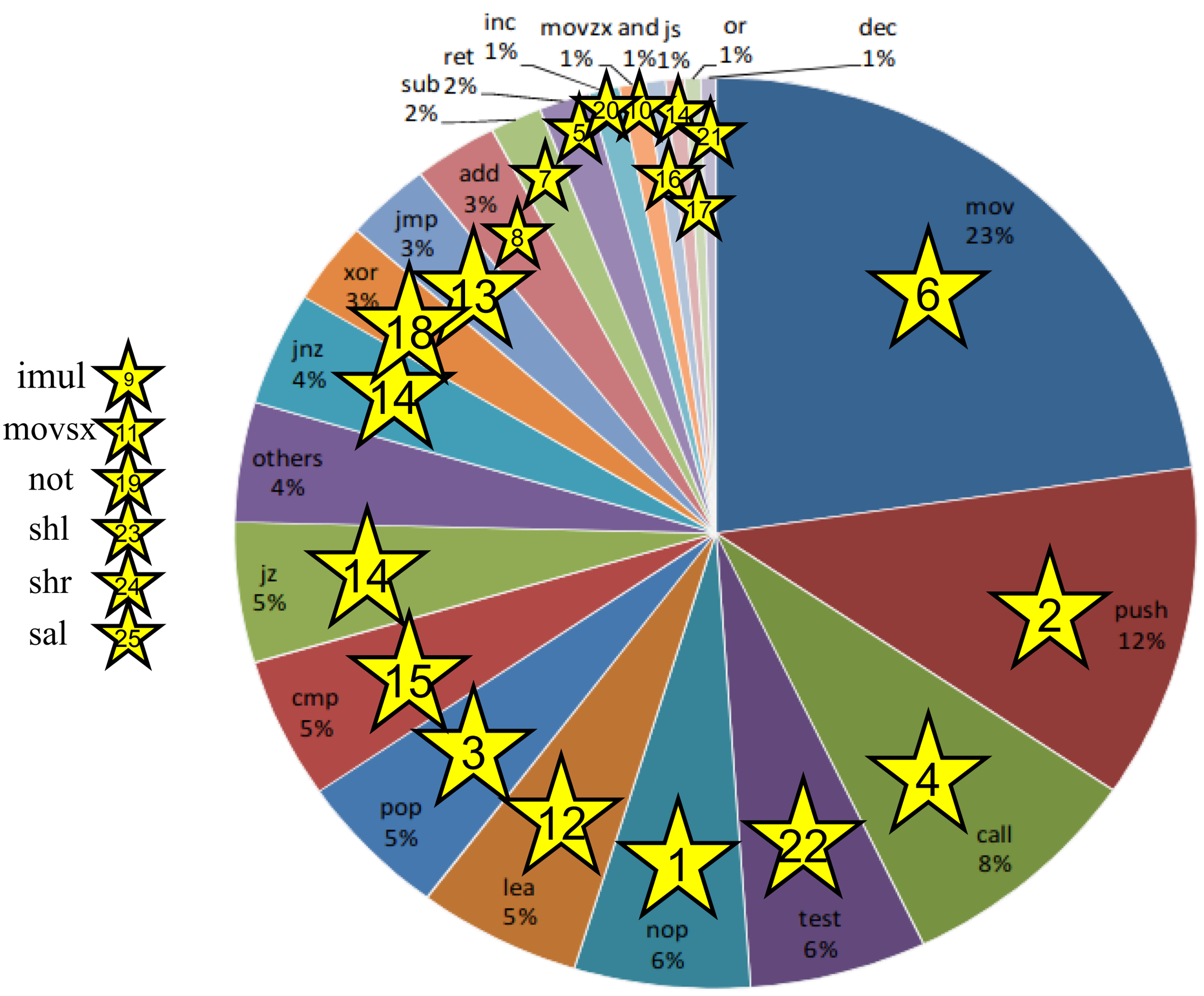


Fig. 2 Web browsers' instruction frequencies

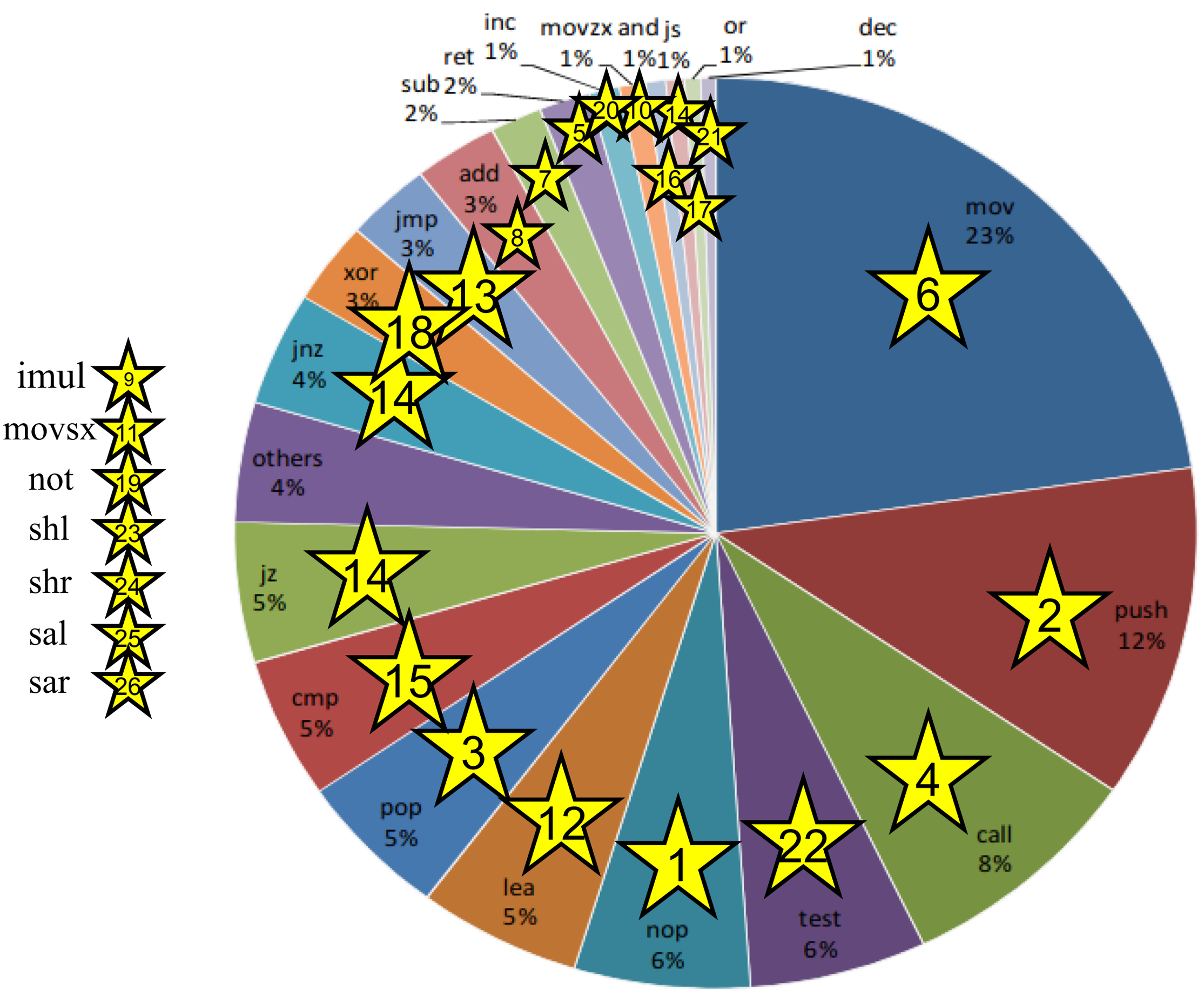


Fig. 2 Web browsers' instruction frequencies